

12

/ 13 LECTURES

Network Troubleshooting

A systematic method plus the six tools of the trade: ping, traceroute, dig, ss, tcpdump and Wireshark — and how to read what they tell you.

OSI Ladder Method

ping & RTT stats

traceroute

nslookup · dig

netstat · ss

TCP States

tcpdump filters

Wireshark

Common Problems

WHAT THIS LECTURE GIVES YOU

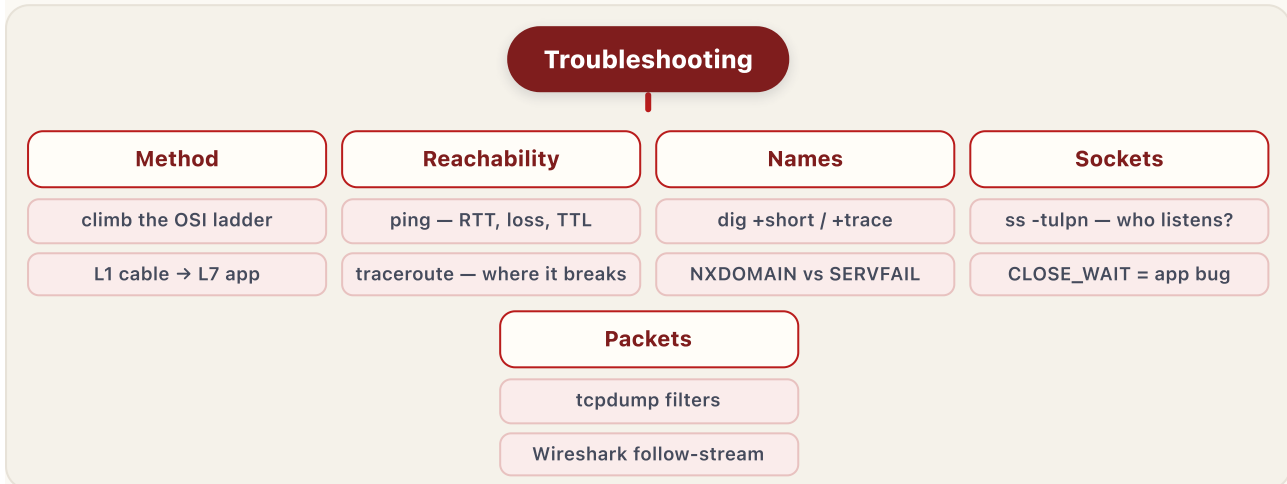
Turn knowledge into diagnostic power: the bottom-up OSI checklist, reading ping statistics like a network engineer, tracing paths and latency jumps, DNS forensics with dig, socket states that expose application bugs, packet-level truth with tcpdump/Wireshark, and a fix-table for the seven problems you'll actually meet.

STUDY-READY EDITION

COMMANDS · OUTPUTS · DIAGNOSES · EXAM PREP

LEARNING OBJECTIVES — AFTER THIS LECTURE YOU CAN

- Apply a **systematic bottom-up** troubleshooting methodology
- Use **ping**, **traceroute**, **dig/nslookup**, **ss/netstat**, **tcpdump**, **Wireshark** correctly
- **Interpret outputs** — latency, loss, TTL, TCP states, retransmissions
- Recognise and fix the **common networking problems**



1 The Method — Climb the OSI Ladder

```
L1 Physical    cable in? Wi-Fi associated? LEDs on?
L2 Data Link   interface up? link light? switch port active?
L3 Network     IP configured? ping gateway? ping 8.8.8.8?
L4 Transport   port open? firewall blocking? service listening?
L7 Application service responding? config right? logs clean?
```

Figure 1. Don't guess — climb. Each rung eliminates a whole class of causes.

THE 6-COMMAND CHECKLIST

TERMINAL

```
1 ip addr          - interface up? has IP?
2 ping <gateway>    - local network OK?
3 ping 8.8.8.8      - Internet routing OK?
4 ping <hostname>   - DNS OK?
5 ss -tulpn | grep <port> - service listening?
6 curl http://localhost:<port> - service responding?
```

2 ping — the Reachability Probe

PING ESSENTIALS

TERMINAL

```
ping google.com    - continuous (Ctrl+C stops)
ping -c 4 google.com - exactly 4 probes
ping -s 1000 google.com - 1000-byte packets (MTU tests)
# Windows: -t continuous · -n 10 for ten
```

```
64 bytes from 142.250.195.46: icmp_seq=1 ttl=115 time=6.82 ms
3 packets transmitted, 3 received, 0% packet loss
rtt min/avg/max/mdev = 6.45/6.79/7.10/0.265 ms
```

Field	Meaning	Field	Meaning
icmp_seq	probe number — gaps = loss	ttl=115	reply TTL → hops ≈ start–received (128–115 = 13 hops — public servers start at 128, not 64)
time	round-trip time	mdev	jitter — high = unstable link
ping result		Likely diagnosis	
Works		L3 connectivity OK to that target	
100% loss		host down, or ICMP deliberately blocked (try a port next!)	
Intermittent loss		congestion · bad cable · Wi-Fi interference	
High avg, low mdev		long distance / VPN — physics, not flakiness	
High mdev		unstable, congested link	
"Name not known"		DNS failure — not a routing problem!	
"Destination Unreachable"		some router has no route to the target	

3 traceroute — Where Does It Hurt?

Sends packets with **TTL = 1, 2, 3...** Each router that kills a packet replies ICMP Time Exceeded — revealing itself. (Windows: `tracert`; better: `mtr` = traceroute + continuous stats.)

```
traceroute google.com · traceroute -T -p 80 google.com (TCP, dodges ICMP blocks)
```

```
1  192.168.1.1      1.0 ms  ← your gateway
2  10.50.0.1        8.2 ms  ← ISP first hop
3  * * *           ← ICMP filtered (traffic still flows!)
4  10.60.100.2     15.1 ms
5  72.14.239.45    18.4 ms  ← Google peering
6  142.250.195.46  19.9 ms  ← destination ✓
```

READING IT **Latency jumps locate congestion:** hop4 at 200 ms after hop3 at 8 ms → the slow link is at hop 4. And * * * does **not** mean broken — many routers just rate-limit ICMP replies while happily forwarding your traffic.

4 dig / nslookup — DNS Forensics

```
nslookup google.com · nslookup google.com 8.8.8.8      — quick checks
dig google.com A · AAAA · MX · NS · TXT                 — record types
dig +short google.com                                   — just the answer
dig +trace google.com                                    — root→TLD→auth walk, live
dig -x 142.250.195.46                                   — reverse PTR
```

Symptom	Diagnosis
NXDOMAIN	domain doesn't exist — check spelling (name vs server problem)
SERVFAIL	DNS server choked — retry with 8.8.8.8 / 1.1.1.1
Works via 8.8.8.8, not via ISP DNS	ISP resolver broken → change DNS config
Wrong IP returned	stale cache or DNS poisoning — flush and re-test

5 ss / netstat — Who Is Listening?

SS — THE MODERN NETSTAT

TERMINAL

```
ss -tulpn          - TCP+UDP, listening, processes, numeric - THE command
ss -t state established - active connections only
ss -t dst :443      - filter by destination port
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	sshd
tcp	LISTEN	0	10	0.0.0.0:8080	0.0.0.0:*	python3
tcp	ESTAB	0	0	192.168.1.5:52341	142.250.195.46:80	curl
udp	UNCONN	0	0	0.0.0.0:53	0.0.0.0:*	dnsmasq

Key fields

0.0.0.0:port = listening on ALL interfaces ·
 127.0.0.1:port = **localhost only — the #1**
"works locally, dead externally" cause! ·
 Recv-Q > 0 = app not reading its data.

TCP states that tell stories

CLOSE_WAIT pile-up → app never calls close() (bug!). **Many TIME_WAIT** → connection churn; enable keep-alive. **Stuck SYN_SENT** → destination down / firewall dropping SYN.

6 tcpdump — Ground-Truth Packets

CAPTURE + FILTERS (BPF SYNTAX)

TERMINAL

```
sudo tcpdump -i eth0 -n          - interface, no name lookups
sudo tcpdump -w cap.pcap         - save for Wireshark
sudo tcpdump host 8.8.8.8        - by host
sudo tcpdump port 443            - by port
sudo tcpdump icmp · udp · tcp    - by protocol
sudo tcpdump host 8.8.8.8 and port 80 - combined
sudo tcpdump -A -n port 80       - ASCII bodies (plain HTTP!)
sudo tcpdump 'tcp[tcpflags] & tcp-syn != 0' - watch handshakes
```

```
12:34:56.123 IP 192.168.1.5.52341 > 142.250.195.46.80: Flags [S], seq 1001
               └─ src ip.port ─┬─ dst ip.port ─┬─ flag ─┘
[S]=SYN  [S.]=SYN-ACK  [.] =ACK  [P.] =data push  [F.] =FIN  [R] =reset
```

SCENARIOS Is my server receiving connections? `tcpdump -n port 8080` · Is DNS alive?
`tcpdump -n port 53` · Seeing [R] where you expected [S.]? — nothing is listening on that port.

7 Wireshark — the Microscope

Feature	How	Why it's magic
Display filters	<code>ip.addr==8.8.8.8 · tcp.port==80 · dns · http</code>	slice captures any way you like, post-capture
Follow TCP Stream	right-click → Follow → TCP Stream	rebuilds the entire conversation (readable HTTP!)
Expert info / retransmissions	<code>tcp.analysis.retransmission</code>	packet loss, out of order — colour-coded
Statistics	Protocol Hierarchy · IO Graph	what protocols dominate? bandwidth over time?

8 Common Problems → Tool → Fix

Problem	Tool	Likely cause	Fix
Can't reach Internet	<code>ping 8.8.8.8</code>	ISP outage / router WAN down	check router, call ISP
DNS broken	<code>dig</code> , <code>nslookup</code>	ISP resolver dead	switch to 8.8.8.8 / 1.1.1.1
Port unreachable	<code>ss -tulpn</code>	service bound to 127.0.0.1, or not running	bind 0.0.0.0 / start service
Random disconnects	<code>mtr</code> , <code>ping -c 100</code>	packet loss mid-path	cables / ISP complaint
Slow website	<code>dig +trace</code> , Wireshark	high TTFB — server slow	DB queries, caching
Many CLOSE_WAIT	<code>ss</code>	app never closes sockets	fix <code>close()</code> in code
Works local, not external	<code>ss</code> , firewall	bind 127.0.0.1 or firewall rule	bind 0.0.0.0 / open port

EXAM *"ping 8.8.8.8 works but ping google.com fails."* → **DNS** (routing is fine — the IP answers; the name doesn't resolve). The single most-asked troubleshooting scenario.

LECTURE 12 — RAPID REVISION CARD

Method: climb OSI L1→L7 — never guess. ip addr → ping GW → ping 8.8.8.8 → ping name → ss → curl.

tracert: TTL-escalation + ICMP Time Exceeded; latency jump = problem hop; * * * ≠ broken.

ss -tulpn: who listens; 127.0.0.1 bind = local-only trap; Recv-Q stuck = app not reading.

tcpdump: host/port/proto filters; -A for HTTP bodies; [S][S.][.][P.][F.][R] flags.

Golden Q: IP pings but name doesn't → DNS. Local works, external doesn't → bind/firewall.

ping: icmp_seq gaps = loss · mdev = jitter · TTL hints hops · "name not known" = DNS.

dig: +short · +trace · -x reverse; NXDOMAIN = bad name, SERVFAIL = server issue.

States: CLOSE_WAIT storm = missing close(); SYN_SENT stuck = unreachable; TIME_WAIT flood = churn.

Wireshark: display filters, Follow TCP Stream, retransmission analysis, IO graphs.

✦ Important Questions — Exam Practice

A. Multiple Choice

Q1. ping 8.8.8.8 succeeds; ping google.com fails. Cause?

MCQ

- a) routing broken
- b) DNS broken
- c) gateway down
- d) firewall

Q2. In traceroute, "* * *" at hop 4 (hop 5 answers) means:

MCQ

- a) hop 4 is down
- b) hop 4 filtered/rate-limited ICMP — traffic still passes
- c) packet loss there
- d) DNS failure

Q3. ss shows your service on 127.0.0.1:8080. Externally it's:

MCQ

- a) reachable
- b) unreachable — localhost bind
- c) reachable via IPv6
- d) cached

Q4. Hundreds of CLOSE_WAIT sockets indicate:

MCQ

- a) healthy server
- b) app not closing sockets
- c) SYN flood
- d) DNS cache miss

Q5. dig returns NXDOMAIN. Meaning?

MCQ

- a) server error
- b) domain doesn't exist
- c) timeout
- d) no IPv6

Q6. Which tcpdump captures only DNS without name resolution?

MCQ

- a) tcpdump port 53 -n
- b) tcpdump udp -v
- c) tcpdump host dns
- d) tcpdump -A port 80

B. Short Answer (2–4 marks)

Q7. State the OSI-ladder methodology and the six-command checklist in order.

SHORT

Q8. Explain how traceroute exploits TTL. Why does it sometimes show * * *?

SHORT

Q9. How do you find the number of hops to a host from ping output alone?

SHORT

Q10. What does Recv-Q > 0 mean? What usually causes it?

SHORT

Q11. Differentiate capture filters and display filters (which tools use which?).

SHORT

Q12. A traceroute shows hop 3 at 8 ms and hop 4 at 200 ms. Where is the congestion and why?

SHORT

C. Long Answer (5–8 marks)

Q13. Diagnose step-by-step: "my website is down" — assume nothing, name the exact command at each ladder rung and what each outcome would prove.

LONG

Q14. Explain the TCP state machine story: LISTEN → SYN_SENT → ESTABLISHED → CLOSE_WAIT → TIME_WAIT, and what each stuck state diagnoses in a real application.

LONG

Q15. Write tcpdump commands: (a) capture DNS to a file without lookups, (b) watch SYN packets to port 80, (c) view plaintext HTTP bodies, (d) capture traffic between two hosts.

CODE

Q16. Match five symptoms from the §8 table to tool, cause and fix — and explain why ICMP blocking can make large transfers hang (connect to PMTUD from Lecture 2).

LONG

ANSWER KEY

A1. b — IP works, name resolution doesn't.

A2. b — routers may rate-limit ICMP; forwarding continues.

A3. b — loopback bind accepts local connections only.

A4. b — remote closed, app never called close().

A5. b — authoritative "no such name".

A6. a — port 53 = DNS; -n disables reverse lookups.

A7. L1→L7 ladder; ip addr → ping GW → ping 8.8.8.8 → ping name → ss -tulpn → curl localhost:port.

A8. TTL=1,2,3... each zeroing router returns Time Exceeded; * * * = filtered/rate-limited ICMP, not failure.

A9. hops $\approx$ sending TTL (64 or 128) – received TTL (e.g. 64–57 = 7).

A10. Data arrived but the application hasn't read it — slow/hung app, or nowhere reading at all.

A11. Capture filters (tcpdump/BPF) limit what's recorded pre-capture; display filters (Wireshark) slice post-capture.

A12. At hop 4 — the jump appears crossing INTO hop 4 (its incoming link), since hop 3 responded quickly.

A13–A16. Ladder + checklist, state stories from §5, the four commands from §6, and PMTUD: blocked "Fragmentation Needed" ICMP = sender never learns to shrink packets.

Wrap-Up — carry these into Lecture 13

THE LADDER Debug **bottom-up**: ip addr → ping GW → ping 8.8.8.8 → dig → curl/port. Each rung isolates one layer's failure.

STATES TELL STORIES SYN_SENT = no reply;
CLOSE_WAIT = you forgot close();
TIME_WAIT = normal 2×MSL cool-down, not a leak.

CAPTURE SMART tcpdump -n (no lookups) to record; Wireshark display filters to analyse. BPF limits what's captured, filters slice what's shown.

SILENT FAILURES Blocked ICMP = PMTUD death: big uploads hang, small ones pass.
traceroute's * * * = filtered ICMP, not always a failure.

NEXT **Lecture 13 — The Complete Internet Journey:** every layer, one packet — the full story of what happens when you press Enter, assembled from all twelve lectures.